MINI PROJECT 14

Menyusun dan Menguji Scheduler Streaming Real-Time

Durasi target:

. Tujuannya mengubah artefak MP13 menjadi kontrak arsitektur streaming yang dapat dipindahkan ke perangkat pada MP15. Gunakan lingkungan Python proyek yang sama. Jangan mengubah model, feature schema, atau split.

Tujuan dan Handoff

Input wajib adalah project_status.json dengan status ready_for_mp14, konfigurasi MP1-MP12, kedua model kandidat, serta hasil MP13. Output wajib adalah realtime_architecture_config.json, timing trace, memory budget proxy, header konstanta C, dan project_status.json dengan status ready_for_mp15.

Input dari MP13	Output MP14	Dipakai MP15 untuk
dua model kandidat	realtime_architecture_config.json	konfigurasi pipeline target
config audio/feature	mp14_realtime_constants.h	konstanta firmware
model/resource proxy	mp14_realtime_metrics.json	baseline profiling
audio training block	mp14_stream_trace.npz	uji scheduler
ready_for_mp14	ready_for_mp15	deployment & quantization

Tabel 14.12. Handoff Mini Project 14.

Prosedur Step by Step

1. [2 menit] Buka folder root running project dan pastikan project_status.json bernilai ready_for_mp14.

2. [2 menit] Pastikan Python environment memiliki numpy, scipy, matplotlib, scikit-learn, joblib, dan torch.

3. [2 menit] Simpan kode berikut sebagai mini_project_14.py di root proyek.

4. [1 menit] Jalankan python mini_project_14.py.

5. [3-5 menit] Periksa output terminal. Pastikan overflow_count = 0 dan status akhir ready_for_mp15.

6. [4 menit] Buka mp14_timing_trace.png. Pastikan mayoritas compute time berada jauh di bawah perioda DMA 10 ms.

7. [4 menit] Buka mp14_memory_budget.png dan catat komponen RAM terbesar. Ingat bahwa ini hanya proxy host.

8. [3 menit] Buka firmware/mp14_realtime_constants.h. Pastikan fs, DMA block, frame, hop, context, dan inference stride sesuai kontrak bab.

9. [2 menit] Isi tabel validasi dan jangan mengubah parameter lagi sebelum memulai MP15.

Kode Python Mini Project 14

Kode berikut adalah skrip lengkap yang telah diuji pada artefak serial MP13. Jalankan dari root proyek. Prediksi label selama simulasi hanya sanity check aliran data; tidak digunakan untuk menilai ulang akurasi atau memilih model.

```
from pathlib import Path
import json, time, shutil
import numpy as np
import matplotlib.pyplot as plt
from scipy.io import wavfile
from scipy.signal import lfilter, lfilter_zi
from scipy.fft import dct
import joblib
import torch
import torch.nn as nn

ROOT = Path('.')
STATUS = ROOT/'project_status.json'
```

```
status = json.loads(STATUS.read_text(encoding='utf-8'))
if status.get('status') != 'ready_for_mp14':
    raise RuntimeError(f"Status harus ready_for_mp14, ditemukan {status.get('status')!r}")

# ---------- Configuration inherited from previous mini projects ----------
project_cfg = json.loads((ROOT/'config/project_config.json').read_text(encoding='utf-8'))
pre_cfg = json.loads((ROOT/'config/preprocessing_config.json').read_text(encoding='utf-8'))
feat_cfg = json.loads((ROOT/'config/feature_config.json').read_text(encoding='utf-8'))
red_cfg = json.loads((ROOT/'config/feature_reduction_config.json').read_text(encoding='utf-8'))
clf_cfg = json.loads((ROOT/'config/classifier_config.json').read_text(encoding='utf-8'))
dl_cfg = json.loads((ROOT/'config/deep_learning_config.json').read_text(encoding='utf-8'))
mp13 = json.loads((ROOT/'results/metrics/mp13_final_test_metrics.json').read_text(encoding='utf-8'))

fs = int(project_cfg['sampling_rate_hz'])
frame_len = int(pre_cfg['frame_length_samples'])
hop_len = int(pre_cfg['hop_length_samples'])
frame_ms = 1000.0 * frame_len / fs
hop_ms = 1000.0 * hop_len / fs
n_mels = int(feat_cfg['mel']['n_mels'])
n_mfcc = int(feat_cfg['mfcc']['n_mfcc'])
context_frames = int(dl_cfg['input_shape_per_sample'][0])
context_ms = frame_ms + (context_frames - 1) * hop_ms
inference_stride_frames = 10   # 100 ms at the running-project hop
inference_stride_ms = inference_stride_frames * hop_ms

dma_block_samples = hop_len
buffer_period_ms = 1000.0 * dma_block_samples / fs
ring_capacity_blocks = 4

# ---------- Helpers ----------
def hz_to_mel(f):
    return 2595.0 * np.log10(1.0 + np.asarray(f) / 700.0)

def mel_to_hz(m):
    return 700.0 * (10.0**(np.asarray(m) / 2595.0) - 1.0)

def make_mel_filterbank(n_fft, fs, n_mels=26, fmin=80.0, fmax=7600.0):
    n_bins = n_fft // 2 + 1
    mel_pts = np.linspace(hz_to_mel(fmin), hz_to_mel(fmax), n_mels + 2)
    hz_pts = mel_to_hz(mel_pts)
    bins = np.floor((n_fft + 1) * hz_pts / fs).astype(int)
    bins = np.clip(bins, 0, n_bins - 1)
    fb = np.zeros((n_mels, n_bins), dtype=np.float64)
    for m in range(1, n_mels + 1):
        left, center, right = bins[m-1], bins[m], bins[m+1]
        center = max(center, left + 1)
        right = max(right, center + 1)
        right = min(right, n_bins - 1)
        for k in range(left, min(center, n_bins)):
            fb[m-1, k] = (k - left) / max(1, center - left)
        for k in range(center, min(right + 1, n_bins)):
            fb[m-1, k] = (right - k) / max(1, right - center)
    return fb

freq = np.fft.rfftfreq(frame_len, d=1.0/fs)
fb = make_mel_filterbank(frame_len, fs, n_mels, float(feat_cfg['mel']['fmin_hz']), min(float(feat_cfg['mel']['fmax_hz']), 0.95*fs/2))
window = np.hamming(frame_len)
lag_min = max(1, int(fs / float(feat_cfg['pitch']['max_hz'])))
lag_max = min(frame_len - 2, int(fs / float(feat_cfg['pitch']['min_hz'])))


def frame_features(frame):
    x = np.asarray(frame, dtype=np.float64)
    x = x - np.mean(x)
    peak = np.max(np.abs(x)) + 1e-12
    x = x * min(1.0, 0.95 / peak)
    alpha = float(pre_cfg['pre_emphasis_alpha'])
    x = np.r_[x[0], x[1:] - alpha*x[:-1]]
    xw = x * window
    rms = float(np.sqrt(np.mean(xw*xw) + 1e-12))
    zcr = float(np.mean(np.abs(np.diff(np.signbit(xw)))))
    X = np.fft.rfft(xw)
    power = np.abs(X)**2 + 1e-15
    psum = np.sum(power) + 1e-15
    centroid = float(np.sum(power * freq) / psum)
    bandwidth = float(np.sqrt(np.sum(power * (freq-centroid)**2) / psum))
    cum = np.cumsum(power)
    idx = int(np.searchsorted(cum, 0.85*cum[-1]))
    rolloff = float(freq[min(idx, len(freq)-1)])
    mel = np.maximum(power @ fb.T, 1e-12)
    logmel = np.log(mel).astype(np.float32)
    mfcc = dct(logmel, type=2, norm='ortho')[:n_mfcc].astype(np.float32)
    f0 = 0.0; conf = 0.0
    ac = np.correlate(xw, xw, mode='full')[frame_len-1:]
    if ac[0] > 1e-12:
        ac = ac/ac[0]
        search = ac[lag_min:lag_max+1]
        if len(search):
            lag = int(np.argmax(search)) + lag_min
            conf = float(ac[lag])
            if conf >= float(feat_cfg['pitch']['confidence_threshold']):
                f0 = float(fs/lag)
    return {'logmel':logmel,'mfcc':mfcc,'rms':rms,'zcr':zcr,'centroid':centroid,
            'bandwidth':bandwidth,'rolloff':rolloff,'f0':f0,'voiced':f0>0}
```

```
def aggregate_39(hist):
  vals=[]
  mfcc=np.stack([h['mfcc'] for h in hist])
  def ms(a, mask=None):
    a=np.asarray(a,dtype=np.float64)
    if mask is not None: a=a[np.asarray(mask,dtype=bool)]
    return (0.0,0.0) if a.size==0 else (float(a.mean()),float(a.std()))
  for j in range(n_mfcc): vals.extend(ms(mfcc[:,j]))
  for key in ['rms','zcr','centroid','bandwidth','rolloff']:
    vals.extend(ms([h[key] for h in hist]))
  voiced=np.array([h['voiced'] for h in hist],dtype=bool)
  vals.extend(ms([h['f0'] for h in hist], voiced))
  vals.append(float(voiced.mean()))
  return np.asarray(vals,dtype=np.float32)

# ---------- Models ----------
class TinyKWSNet(nn.Module):
  def __init__(self, n_classes):
    super().__init__()
    self.features = nn.Sequential(
      nn.Conv2d(1,8,3,padding=1,bias=False), nn.BatchNorm2d(8), nn.ReLU(), nn.MaxPool2d(2),
      nn.Conv2d(8,16,3,padding=1,bias=False), nn.BatchNorm2d(16), nn.ReLU(), nn.AdaptiveAvgPool2d((1,1)))
    self.classifier = nn.Linear(16,n_classes)
  def forward(self,x):
    return self.classifier(self.features(x).flatten(1))

classic_model = joblib.load(ROOT/clf_cfg['model_file'])
ckpt = torch.load(ROOT/dl_cfg['model_file'], map_location='cpu', weights_only=False)
dl_classes = list(ckpt['classes']); mu=float(ckpt['normalization']['mean']); sigma=float(ckpt['normalization']['std'])
tiny = TinyKWSNet(len(dl_classes)); tiny.load_state_dict(ckpt['state_dict']); tiny.eval()
selected=np.asarray(red_cfg['selected_indices'],dtype=int)
scaler_mean=np.asarray(red_cfg['scaler_mean'],dtype=np.float32)
scaler_scale=np.asarray(red_cfg['scaler_scale'],dtype=np.float32)

# Warm-up inference runtimes.
with torch.no_grad():
  dummy=torch.zeros((1,1,context_frames,n_mels),dtype=torch.float32)
  for _ in range(5): tiny(dummy)
for _ in range(5): classic_model.predict(np.zeros((1,len(selected)),dtype=np.float32))

# ---------- Input stream ----------
source = ROOT/'data/raw/mp09_dataset/nyala_b01.wav'  # training block; no reuse of test for architecture design
sr, audio = wavfile.read(source)
if sr != fs: raise RuntimeError(f'sampling rate {sr} != {fs}')
if audio.ndim>1: audio=audio[:,0]
if np.issubdtype(audio.dtype,np.integer):
  audio=audio.astype(np.float64)/max(abs(np.iinfo(audio.dtype).min),np.iinfo(audio.dtype).max)
else: audio=audio.astype(np.float64)
# Repeat to obtain several inference opportunities without collecting new data.
audio=np.tile(audio,4)

# Final filter from MP6 as a stateful streaming filter.
filt=np.load(ROOT/status['filter_final_coefficients'],allow_pickle=True)
b=np.asarray(filt['b'],dtype=np.float64); a=np.asarray(filt['a'],dtype=np.float64)
zi=lfilter_zi(b,a)*0.0

pcm=[]; feature_hist=[]
trace=[]; classic_ms=[]; cnn_ms=[]; labels_trace=[]
virtual_cpu_ready=0.0
frame_counter=0
n_infer=0

for bi, start in enumerate(range(0,len(audio),dma_block_samples)):
  block=audio[start:start+dma_block_samples]
  if len(block)<dma_block_samples: break
  arrival_ms=bi*buffer_period_ms
  t0=time.perf_counter()
  y,zi=lfilter(b,a,block,zi=zi)
  pcm.extend(y.tolist())
  if len(pcm)>frame_len: pcm=pcm[-frame_len:]
  did_infer=False; pred_c=''; pred_d=''
  if len(pcm)==frame_len:
    ff=frame_features(np.asarray(pcm))
    feature_hist.append(ff)
    if len(feature_hist)>context_frames: feature_hist=feature_hist[-context_frames:]
    frame_counter += 1
    if len(feature_hist)==context_frames and (frame_counter-context_frames)%inference_stride_frames==0:
      did_infer=True; n_infer += 1
      full=aggregate_39(feature_hist)
      x8=((full-scaler_mean)/scaler_scale)[selected][None,:]
      tc=time.perf_counter(); pred_c=str(classic_model.predict(x8)[0]); classic_ms.append((time.perf_counter()-tc)*1000)
      lm=np.stack([h['logmel'] for h in feature_hist]).astype(np.float32)
      x=torch.from_numpy(((lm-mu)/sigma)[None,None,...]).float()
      td=time.perf_counter()
      with torch.no_grad(): pred_d=dl_classes[int(tiny(x).argmax(1).item())]
      cnn_ms.append((time.perf_counter()-td)*1000)
      labels_trace.append((bi,pred_c,pred_d))
  compute_ms=(time.perf_counter()-t0)*1000
  start_ms=max(arrival_ms,virtual_cpu_ready)
  finish_ms=start_ms+compute_ms
  waiting_ms=start_ms-arrival_ms
  queue_blocks=int(np.floor(waiting_ms/buffer_period_ms+1e-12))
```

```
  overflow=queue_blocks>=ring_capacity_blocks
  virtual_cpu_ready=finish_ms
  trace.append([bi,arrival_ms,compute_ms,waiting_ms,queue_blocks,int(did_infer),int(overflow)])

trace=np.asarray(trace,dtype=np.float64)
# Memory proxy. MP15 will measure real target memory.
dma_bytes=2*dma_block_samples*2
pcm_ring_bytes=frame_len*2
logmel_ring_bytes=context_frames*n_mels*4
classical_frame_state_bytes=context_frames*(13+7)*4
# Float32 activation upper bound: first conv output 8*T*F before pooling.
activation_proxy_bytes=8*context_frames*n_mels*4
cnn_weight_bytes=int(mp13['resource_proxy']['tiny_cnn']['float32_parameter_bytes'])
classical_serialized=int(mp13['resource_proxy']['classical']['serialized_bytes'])

metrics={
 'source_audio':str(source),
 'stream_duration_s':float(len(audio)/fs),
 'sample_rate_hz':fs,
 'dma_block_samples':dma_block_samples,
 'dma_block_ms':buffer_period_ms,
 'double_buffer':True,
 'ring_capacity_blocks':ring_capacity_blocks,
 'frame_samples':frame_len,
 'frame_ms':frame_ms,
 'hop_samples':hop_len,
 'hop_ms':hop_ms,
 'context_frames':context_frames,
 'context_ms':context_ms,
 'inference_stride_frames':inference_stride_frames,
 'inference_stride_ms':inference_stride_ms,
 'n_blocks':int(len(trace)),
 'n_inferences':int(n_infer),
 'mean_block_compute_ms':float(trace[:,2].mean()),
 'p95_block_compute_ms':float(np.percentile(trace[:,2],95)),
 'max_block_compute_ms':float(trace[:,2].max()),
 'max_waiting_ms':float(trace[:,3].max()),
 'max_queue_blocks':int(trace[:,4].max()),
 'overflow_count':int(trace[:,6].sum()),
 'realtime_condition_pass':bool(trace[:,6].sum()==0 and np.percentile(trace[:,2],95)<buffer_period_ms),
 'classical_inference_ms_mean':float(np.mean(classic_ms) if classic_ms else 0),
 'classical_inference_ms_p95':float(np.percentile(classic_ms,95) if classic_ms else 0),
 'tiny_cnn_inference_ms_mean':float(np.mean(cnn_ms) if cnn_ms else 0),
 'tiny_cnn_inference_ms_p95':float(np.percentile(cnn_ms,95) if cnn_ms else 0),
 'memory_proxy_bytes':{
  'dma_double_buffer_int16':dma_bytes,
  'pcm_frame_ring_int16':pcm_ring_bytes,
  'logmel_context_float32':logmel_ring_bytes,
  'classical_frame_state_float32':classical_frame_state_bytes,
  'tiny_cnn_activation_float32_upper_proxy':activation_proxy_bytes,
  'tiny_cnn_weights_float32':cnn_weight_bytes,
  'classical_serialized_model':classical_serialized,
 },
 'interpretation':'Host-side scheduling and memory proxy only. Target-device latency, peak RAM, flash, and power must be measured in MP15.',
 'deployment_model_selected':False,
 'deployment_selection_deferred_to':'MP15'
}

config={
 'sample_rate_hz':fs,
 'input_interface':'I2S_or_ADC_to_DMA',
 'dma_block_samples':dma_block_samples,
 'dma_buffer_count':2,
 'dma_block_ms':buffer_period_ms,
 'pcm_ring_samples':frame_len,
 'feature_hop_samples':hop_len,
 'feature_frame_samples':frame_len,
 'feature_context_frames':context_frames,
 'feature_context_ms':context_ms,
 'inference_stride_frames':inference_stride_frames,
 'inference_stride_ms':inference_stride_ms,
 'scheduler':'DMA ISR only signals buffer-ready; DSP/feature/inference run outside ISR',
 'candidate_models':['mp11_classical_baseline','mp12_tiny_cnn'],
 'final_model_selection':'DEFERRED_TO_MP15',
 'overflow_policy':'increment_counter_and_drop_oldest_unprocessed_block',
 'instrumentation':['frame_counter','overflow_counter','timestamp_us','stage_latency_us']
}

(ROOT/'config').mkdir(exist_ok=True)
(ROOT/'results/metrics').mkdir(parents=True,exist_ok=True)
(ROOT/'results/figures').mkdir(parents=True,exist_ok=True)
(ROOT/'features/intermediate').mkdir(parents=True,exist_ok=True)
(ROOT/'firmware').mkdir(parents=True,exist_ok=True)
(ROOT/'config/realtime_architecture_config.json').write_text(json.dumps(config,indent=2),encoding='utf-8')
(ROOT/'results/metrics/mp14_realtime_metrics.json').write_text(json.dumps(metrics,indent=2),encoding='utf-8')
np.savez_compressed(ROOT/'features/intermediate/mp14_stream_trace.npz', trace=trace,
                    columns=np.array(['block','arrival_ms','compute_ms','waiting_ms','queue_blocks','did_infer','overflow'],dtype=object))

# Header constants for MP15 firmware integration.
header=f'''#pragma once\n// Auto-generated by Mini Project 14\n#define AUDIO_FS_HZ {fs}\n#define DMA_BLOCK_SAMPLES {dma_block_samples}\n#define
DMA_BUFFER_COUNT 2\n#define AUDIO_FRAME_SAMPLES {frame_len}\n#define FEATURE_HOP_SAMPLES {hop_len}\n#define FEATURE_CONTEXT_FRAMES
{context_frames}\n#define INFERENCE_STRIDE_FRAMES {inference_stride_frames}\n#define AUDIO_RING_CAPACITY_BLOCKS {ring_capacity_blocks}\n'''
(ROOT/'firmware/mp14_realtime_constants.h').write_text(header,encoding='utf-8')
```

```
# Figures
fig,ax=plt.subplots(figsize=(8.2,4.2))
ax.plot(trace[:,1],trace[:,2],lw=1,label='compute per DMA block')
ax.axhline(buffer_period_ms,ls='--',lw=1,label='DMA block period')
ax.set_xlabel('Stream time (ms)'); ax.set_ylabel('Host compute time (ms)'); ax.grid(alpha=.2); ax.legend();
ax.set_title('MP14 host-side real-time scheduling trace'); fig.tight_layout(); fig.savefig(ROOT/'results/figures/mp14_timing_trace.png',dpi=170); plt.close(fig)

mem=metrics['memory_proxy_bytes']; names=list(mem.keys()); vals=np.array(list(mem.values()),dtype=float)/1024.0
fig,ax=plt.subplots(figsize=(8.0,4.4)); ax.barh(np.arange(len(names)),vals); ax.set_yticks(np.arange(len(names))); ax.set_yticklabels([n.replace('_',' ') for n in
names],fontsize=8); ax.set_xlabel('KiB'); ax.set_title('MP14 memory budget proxy'); ax.grid(axis='x',alpha=.2); fig.tight_layout();
fig.savefig(ROOT/'results/figures/mp14_memory_budget.png',dpi=170); plt.close(fig)

# Update project status.
status['status']='ready_for_mp15'; status['last_completed_mini_project']=14
status['mp14']={
 'realtime_config':'config/realtime_architecture_config.json',
 'stream_trace':'features/intermediate/mp14_stream_trace.npz',
 'metrics':'results/metrics/mp14_realtime_metrics.json',
 'firmware_constants':'firmware/mp14_realtime_constants.h',
 'timing_figure':'results/figures/mp14_timing_trace.png',
 'memory_figure':'results/figures/mp14_memory_budget.png',
 'deployment_candidates':['models/mp11_classical_baseline.joblib','models/mp12_tiny_cnn_state.pt'],
 'deployment_selection_deferred_to':'MP15'
}
STATUS.write_text(json.dumps(status,indent=2),encoding='utf-8')

print('MP14 selesai')
print('blocks:',len(trace),'inferences:',n_infer)
print('p95 block compute ms:',round(metrics['p95_block_compute_ms'],4),'period ms:',buffer_period_ms)
print('max queue blocks:',metrics['max_queue_blocks'],'overflow:',metrics['overflow_count'])
print('classical infer mean ms:',round(metrics['classical_inference_ms_mean'],4))
print('tiny cnn infer mean ms:',round(metrics['tiny_cnn_inference_ms_mean'],4))
print('status:',status['status'])
```

Kriteria Mini Project 14

Pemeriksaan	Kriteria lulus
Status awal	ready_for_mp14
DMA period	10 ms
Context	96 frame / 975 ms
Inference stride	100 ms
Overflow count	0 pada simulasi host
Realtime condition	True
Model selection	False / deferred to MP15
Header C	mp14_realtime_constants.h tersedia
Status akhir	ready_for_mp15

Tabel 14.13. Checklist kelulusan Mini Project 14.

Artefak yang wajib dibawa ke Mini Project 15

Jangan menghapus config/realtime_architecture_config.json, firmware/mp14_realtime_constants.h, features/intermediate/mp14_stream_trace.npz, results/metrics/mp14_realtime_metrics.json, kedua model kandidat MP11/MP12, serta seluruh konfigurasi preprocessing dan feature. MP15 akan melakukan deployment, quantization, numeric-equivalence check, dan benchmark perangkat berdasarkan kontrak ini.

Referensi

1. Lyons, R. G. (2011). Understanding Digital Signal Processing, 3rd ed. Pearson.

2. Oppenheim, A. V., & Schafer, R. W. (2010). Discrete-Time Signal Processing, 3rd ed. Pearson.

3. Rabiner, L. R., & Schafer, R. W. (2010). Theory and Applications of Digital Speech Processing. Pearson.

4. Warden, P., & Situnayake, D. (2019). TinyML. O'Reilly Media.

5. Sze, V., Chen, Y.-H., Yang, T.-J., & Emer, J. S. (2020). Efficient Processing of Deep Neural Networks. Morgan & Claypool.

6. ARM. CMSIS-DSP Software Library documentation. Conceptual reference for optimized DSP kernels on Arm Cortex processors.

7. TensorFlow Lite for Microcontrollers documentation. Conceptual reference for embedded inference runtime and memory planning.

8. FreeRTOS documentation. Conceptual reference for task scheduling, queues, and real-time instrumentation.